Retrieval-Augmented Generation Systems at Scale: Architecture, Challenges, and Future Directions

Mathis Doutre

Abstract

Retrieval-Augmented Generation (RAG) [1] combines large language models (LLMs) with external retrieval to ground generation in factual data. By retrieving relevant documents at query time, RAG reduces hallucinations and the problem of outdated model knowledge without needing frequent retraining.

However, deploying RAG at scale is fundamentally a Big Data systems challenge. It requires indexing and searching large collections of high-dimensional embeddings under latency, freshness, and cost constraints. As the corpus grows and data changes continuously, maintaining efficient retrieval and up-to-date knowledge becomes increasingly complex.

This paper examines RAG systems through the lens of large-scale data systems and engineering. We present base architectures, vector indexing and retrieval techniques, and practical challenges such as document chunking, embedding lifecycle management, hybrid retrieval, and cost optimization. We analyze key limitations (such as stale knowledge, access control, and weak evaluation practices) and discuss emerging directions like knowledge-graph integration and automated index refresh.

1 Introduction and Motivation

Retrieval-Augmented Generation (RAG) augments large language models with an explicit retrieval step that grounds generation in external data.

1.1 Why RAG Exists

Large language models are prone to hallucinations: they may generate fluent but incorrect statements based on patterns learned from static training data [2]. Once trained, their knowledge is also frozen at a fixed cutoff, making them unaware of new events [3]. As a result, standalone LLMs can produce outdated or fabricated answers [4]. RAG was created to tackle these problems. By inserting a retrieval step, RAG allows an LLM to gather some information from an external knowledge source (documents, databases, etc.) before generating an answer. The model conditions its output on retrieved documents rather than relying on parametric memory. This reduces hallucinations and incorrect answers because the LLM can ground its

response in real data rather than guessing [1]. For time-sensitive queries, a RAG system can retrieve recent documents and incorporate them directly into the generation process. RAG also removes the need for constant retraining of LLMs on new data, the system's knowledge can be refreshed simply by updating the document index.

1.2 Why RAG Is a Big Data Systems Problem

While RAG is frequently framed as a modeling technique, deploying it at scale is fundamentally a systems problem. The reason is scale: real-world knowledge databases can contain millions or billions of documents that the RAG system must index and search through in real-time. Storing and searching this volume of information requires robust data infrastructure. Each document in a RAG system is converted into a high-dimensional embedding vector, often with thousands of dimensions, to enable semantic search [5]. Managing these embeddings demands indexing structures and memory management. A naive approach would be infeasible.

Thus, specialized vector indexes and databases (discussed in Section 4) are employed to achieve sub-linear search time. Moreover, a RAG application (like a enterprise QA chatbot) may be used by thousands of users at the same time, so the system must handle many queries without slowing down. Achieving low-latency responses (on the order of a second or two per query) under these loads is non-trivial and involves careful engineering (networking, caching, and parallel processing). A second challenge is continuous ingestion and index maintenance as data changes. In dynamic domains (news, finance, etc.), new content arrives constantly. A RAG system must continuously incorporate fresh documents and remove or update outdated ones, essentially performing streaming updates to its index. This requires pipelines for real-time embedding of new data and strategies to refresh or rebuild indexes without service downtime. Balancing index freshness with computational cost is a real challenge.

At scale, storage and inference costs become a primary design constraint, particularly when embeddings must be kept in memory or external LLM APIs are used per query. This forces system designers to consider trade-offs between compute, memory, and accuracy [6].

1.3 Target Audience

This review is written for an audience of computer science graduate students and data systems professionals who are interested in the systems and architecture side of AI applications. We assume readers have a general understanding of machine learning and databases. The goal is to provide sufficient background (Section 2) so that non-experts in LLMs can grasp the technical discussions. We aim to keep explanations accessible. For instance, we explain key concepts like vector indexes and embedding models in simple terms, but we also explain design trade-offs and future directions that advanced readers will appreciate. By the end, readers should understand why RAG is important, how it works at scale, and what are the main challenges.

2 Background and Fundamentals

Before diving into system architectures and advanced techniques, we cover the foundational background of retrieval augmented generation. This section provides context on RAGs, outlines the basic pipeline, and explains why naive implementations do not work at scale.

2.1 From Information Retrieval (IR) to Neural Retrieval

RAG combines simple information retrieval techniques with neural embedding-based retrieval. In trivial IR systems (e.g., search engines like Lucene/Elasticsearch), documents are retrieved using sparse methods such as TF-IDF or BM25. These represent text as high-dimensional vectors where each dimension corresponds to a term (word) in the vocabulary, and non-zero values indicate term frequencies [7]. Sparse vectors focus mainly on exact keyword matching. For example, a search query "machine learning" would retrieve documents containing those exact words, and ranking can depend on term frequency. Sparse retrieval excels when exact terminology matters (e.g., legal search for a specific phrase). However, it has well-known limitations: it struggles with synonyms or concept matches (a query for "car" won't match a doc with "automobile"), and it can fail on longer natural language queries where the exact words might differ [7].

Neural retrieval (dense retrieval) has been created to overcome these limitations. Using neural language models (like BERT or sentence transformers), we can encode texts into dense embedding vectors (continuous vectors in a lower-dimensional space that capture semantic similarity). With dense vectors, even if two texts have no words in common, if they are semantically close the vectors will be close in distance. For instance, an embedding for "AI algorithms" might be near the embedding of "neural networks in computing".

It's ideal for open-ended queries often seen in QA or chatbot scenarios. The trade-off is that dense models require computation (both to train and to query) and large vector storage. In fact, combining sparse and dense results is increasingly common to leverage the precision of keyword matching and the flexibility of semantic matching [7]. We'll revisit hybrid approaches in Section 6.2. Overall, retrieval has shifted from exact term matching toward embedding-based similarity, which underlies modern RAG systems.

2.2 Basic RAG Pipeline

At a high level, a Retrieval-Augmented Generation system can be seen as a pipeline with stages. The basic RAG pipeline usually has the following steps:

- Document Ingestion: The system collects knowledge from documents, webpages, or structured data.
- Chunking and Embedding: Documents are split into chunks and encoded into dense embedding vectors, which are stored inside a vector database.
- Retrieval: The user query is embedded, and used to retrieve the top-k most similar chunks from the vector index.
- Generation: Retrieved chunks are combined with the query and passed to an LLM to

generate a grounded answer.

- Post-Processing (Optional): Formatting, citation attachment, or lightweight validation.

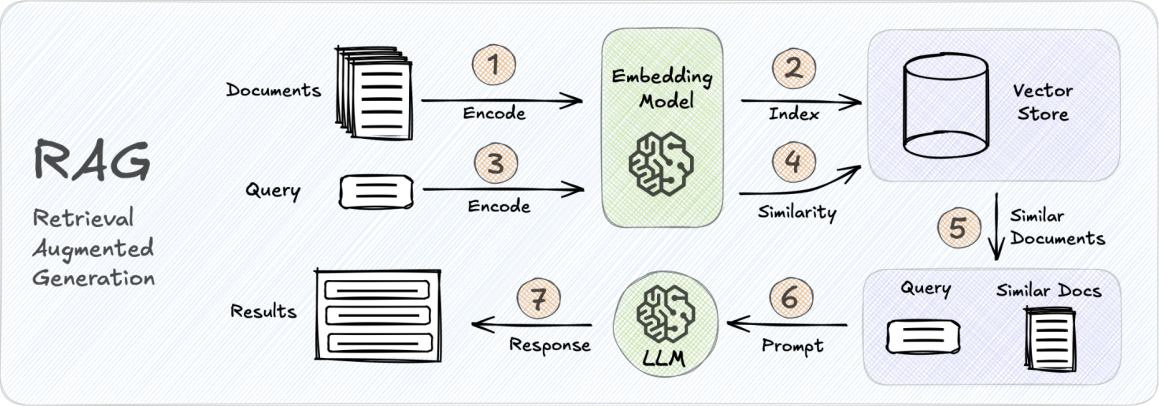

Figure 1: Basic RAG pipeline. Documents are encoded into embeddings and indexed in a vector store. At query time, the user query is embedded, similar documents are retrieved via vector similarity search, and the retrieved context is injected into the prompt for the large language model (LLM) to generate a grounded response.

2.3 Why Naïve RAG Fails at Scale

One can relatively easily prototype a basic RAG system on a small scale. However, naïve implementations become inefficient at scale due to several compounding factors:

- Latency : A naive approach would compare a query's embedding against every document vector in the corpus. This linear scan becomes too slow as the number of vectors grows. The curse of dimensionality means that efficient indexing is needed.
- Memory and Storage Bottlenecks: Each document embedding might be hundreds of floats. Millions of those can occupy many gigabytes of memory. A naive approach might try to keep all embeddings in RAM for speed, but at some scale, this becomes impractical or extremely expensive. Also, storing everything on disk without careful indexing would slow retrieval. Naively scaling up could hit hardware limits. It becomes essential to use techniques like vector compression (e.g., product quantization) or disk-based indices to manage this, which we discuss in Section 4.2.
- Stale Knowledge and Index Maintenance: Another possible failure is letting the knowledge base get stale. In a deployed system, data is not static, new documents are created, information changes. A naive RAG setup might not include a strategy for continuous updates, meaning the system's answers become stale. Handling this at scale is non-trivial: re-embedding and re-indexing a billion-doc corpus daily is extremely costly. Many production systems update incrementally or maintain a schedule.
- Operational Complexity: At scale, even "simple" things like deployment and monitoring become difficult, we likely need to split the workload across multiple servers (for the vector DB, the LLM service, etc.), introducing distributed systems issues.

3 System Architecture of RAG at Scale

We now focus on how to architect a RAG system that can operate at scale. We will describe a typical architecture, distinguish between offline and online components, and discuss different deployment patterns.

3.1 End-to-End Architecture

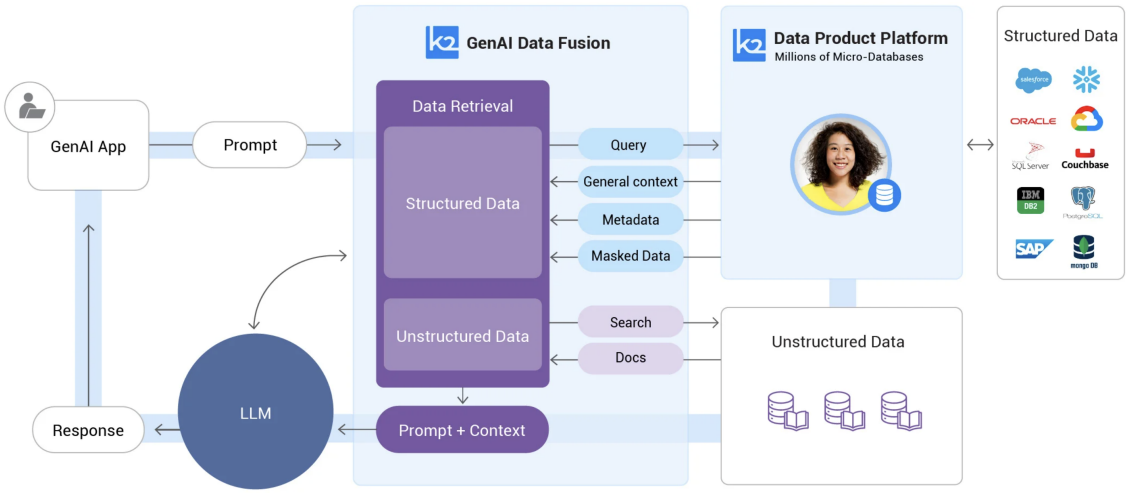

Figure 2: An end-to-end Retrieval-Augmented Generation (RAG) system architecture. The retrieval component connects user queries with enterprise data sources, feeding relevant information into the large language model (LLM) for answer generation. Source: k2view.com.

In production, RAG systems are usually deployed as distributed services with separate retrieval and generation components. The components involved might include:

- Data Sources: The repositories of content that RAG will draw from. They can be internal databases, document stores, cloud storage buckets, web crawls, etc.
- Ingestion and Embedding Pipeline: This is an offline component responsible for taking raw data from the sources, preprocessing it, and generating embeddings for each chunk. This pipeline might be distributed. The pipeline writes the resulting vectors into the vector store. It should also assign IDs, handle duplicates, and so on. Often, this pipeline is continuously running or triggered periodically to pick up new data.
- Vector Store / Index: The heart of retrieval is a vector database. This could be a standalone vector DB (like Milvus, Pinecone, etc.) or a custom service using libraries like FAISS. The vector store holds all document embeddings and provides an API to perform similarity search (given a query embedding, return the top-k closest vectors).The vector store is likely distributed across multiple nodes. It might partition the index across shards and replicate for fault-tolerance (more on this in Section 4.3).
- Retriever Service: Usually, there is a retriever service that joins the interaction between the query and the vector store. When a query comes in, this service takes the user query,

uses an embedding model to encode it into a vector, then calls the vector store's search API to get results. It may then post-process the results (e.g., filter or re-rank them) and package the retrieved texts. It basically encapsulates the retrieval logic. In some designs, the retriever is merged with the vector store (the store does the embedding too), but splitting them offers some flexibility (you can update embedding models independently, etc.).

- LLM (Generation) Service: The large language model itself is usually served behind an API, like an external API (OpenAI, etc.) or a self hosted LLM. The architecture includes a generator service that accepts the user's query along with retrieved context and then produces the final answer via the LLM. This service handles prompt construction (formatting the question + documents into a prompt template) and calls the LLM. If using an open-source model, this might run on GPU servers. If using a cloud API, the service just calls that API. The generation component can also incorporate prompt engineering and some guardrails as needed.
- Orchestration : There may be an orchestrator in the application backend that coordinates the steps. For example, a user query hitting an API endpoint will trigger calls to the retriever service, wait for the results, then call the LLM service, then return the answer. In complex setups, a workflow engine or maybe a chain-of-thought agent might manage multi-step interactions, but for a basic Q&A it's a simple sequential orchestrator. This layer also can handle user context (sessions), authentication, and format the output (e.g., attaching source links).

3.2 Offline vs Online Components

It is important to separate **offline** and **online** components in a RAG system, since they have very different performance and design constraints.

Offline components are all preprocessing steps that do not happen during the user query. This covers document ingestion, cleaning, embedding generation, and index construction. These jobs are often run in batch or continuously. Offline processes can be slow and compute-heavy, since their latency does not affect users directly. Examples include rebuilding the index after adding new documents, recreating embeddings when the embedding model changes, or precomputing structures such as ANN graphs. The goal of offline work is to prepare data so that query-time execution is fast.

Online components are executed when a user submits a query. This contains creating the query embedding, running vector search, and generating the final answer with an LLM. These steps have to be low-latency, be scalable and reliable. Online services are usually replicated and optimized for throughput, since it directly impacts user experience.

This separation enables different optimizations: **offline stages can trade time for quality, while online stages often trade some accuracy for speed**. A well-designed RAG system pushes as much work as possible offline, so that online serving is lightweight and fast. In practice, some systems use a hybrid setup (offline base index plus online incremental updates) to balance freshness and efficiency.

3.3 Deployment Architectures

When deploying a RAG system at scale, we must decide how to structure services, infrastructure, and hardware. Design axes include : monolithic vs microservice architectures ; cloud vs on-premises deployment ; and CPU vs GPU usage.

Monolithic vs Microservices. A monolithic design does retrieval and generation into a single application, which is simple to build but harder to scale. Usually, large RAG systems favor microservices, where the vector index, embedding model, and LLM are deployed as separate services communicating via APIs. This allows independent scaling (e.g., adding more index nodes or GPU servers) and better specialization of resources.

Cloud vs On-Premises. Cloud deployment is common due to scaling and access to managed services (e.g., hosted vector databases or model serving). This is useful when workloads change. Private cloud deployments are often chosen for data privacy or cost control at steady scale. Hybrid architectures also exist, combining on-prem retrieval with cloud-based LLMs. The choice depends on our data sensitivity, network constraints, and cost considerations.

GPU vs CPU Trade-offs. Hardware selection is critical, especially for the LLM. Large-scale LLMs requires GPUs due to the computational cost of matrix operations, while vector search and orchestration often run efficiently on CPUs. A common setup uses GPUs only for LLM inference, and CPUs elsewhere to control costs.

In practice, a scaled RAG system often consists of microservices deployed on Kubernetes, with CPU-based vector databases and GPU-backed LLM services, running either in the cloud or on-prem.

4 Vector Databases and Indexing Techniques

Vector databases enable efficient storage and retrieval of high-dimensional embedding vectors. At scale specialized similarity search and indexing techniques essential.

We discuss the challenges of high-dimensional similarity search, common approximate nearest neighbor (ANN) methods such as HNSW and IVF, and how vector databases distribute data to support large-scale and low-latency queries. We also briefly review widely used implementations, including FAISS, Milvus, Pinecone, and Weaviate.

4.1 High-Dimensional Similarity Search

High-dimensional similarity search is the core operation behind retrieval in RAG systems. Given a query embedding, the goal is to find the nearest vectors among millions or billions of stored embeddings. A naive exact search requires computing distances to all vectors, which is $O(N)$ and infeasible at scale. Traditional spatial indexes (e.g., KD-trees) also break down in high dimensions due to the curse of dimensionality, effectively degenerating to linear scan [8].

To address this problem, modern systems rely on **Approximate Nearest Neighbor (ANN)** algorithms. ANN methods trade exactness for speed by returning near-optimal neighbors with high probability [9]. Retrieval quality is typically measured by recall. In RAG, slightly

imperfect recall is usually acceptable as long as relevant documents are retrieved, since the LLM can tolerate some noise.

ANN techniques used in practice fall into a some main categories. **Graph-based methods** (e.g., HNSW) organize vectors into a navigable graph and perform greedy search, offering strong recall and stable latency. **Partitioning and quantization methods** (e.g., IVF, Product Quantization) cluster or compress vectors to reduce the search space and speed up distance computations. **Hash- or tree-based methods** (e.g., LSH) exist as well, but are less common in modern large-scale systems due to weaker recall or higher memory costs.

A key design trade-off is between recall, latency, and memory usage. Most ANN indexes have tunable parameters (such as search depth or number of clusters probed) that allows to choose an appropriate point on the speed–accuracy curve. At scale, predictable tail latency is especially important, which is one reason graph-based methods like HNSW are often adopted.

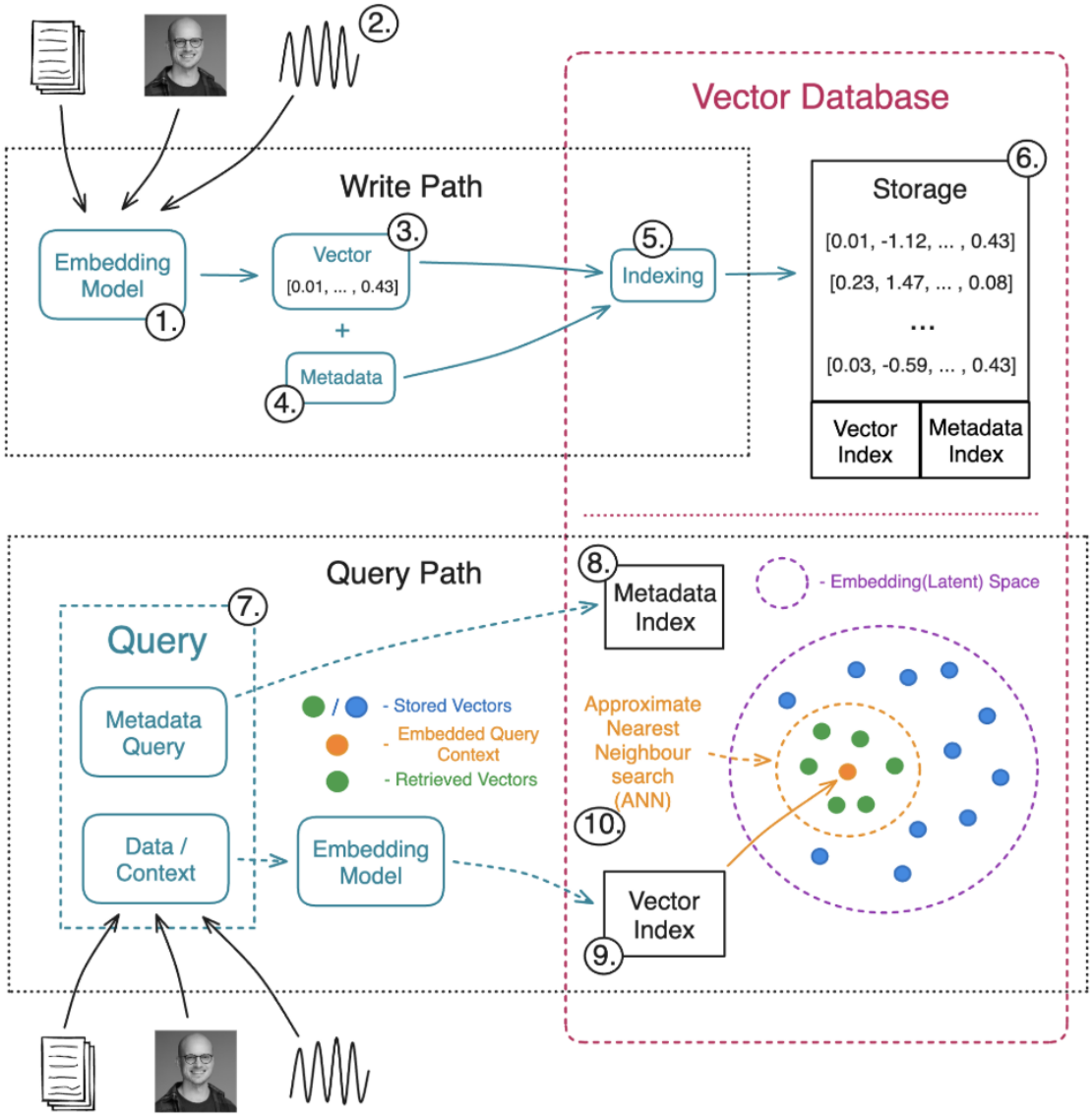

Figure 3: Architecture of a vector database showing the write path and query path. During ingestion, documents are embedded, enriched with metadata, and indexed. During querying, the embedded query is matched against the vector index using approximate nearest neighbor (ANN) search to retrieve semantically similar vectors. Metadata filtering and vector similarity jointly determine the final results.

4.2 ANN Indexing Methods

Modern RAG systems rely on specialized ANN indexes to enable fast similarity search at scale. The most widely used methods include graph-based, partitioning/quantization-based, and disk-based approaches.

Hierarchical Navigable Small World (HNSW). HNSW is a graph-based ANN method that organizes vectors into a hierarchy of proximity graphs [10]. Upper layers contain fewer

nodes with long-range links, while lower layers provide dense local connectivity. Search proceeds top-down: starting from a high-level entry point, the algorithm greedily moves closer to the query and descends layer by layer until reaching the full graph. This structure enables fast, almost-logarithmic search with high recall. HNSW is widely used in vector databases, though it has relatively high memory overhead due to stored graph edges.

Inverted File Index (IVF) and Product Quantization (PQ). IVF partitions the vector space into clusters (usually via k-means) and stores vectors in inverted lists per centroid. At query time, only a subset of clusters is searched, significantly reducing comparisons. PQ further compresses vectors into compact codes, enabling faster distance computation and reduced memory usage. The combined IVF–PQ approach, popularized by FAISS [11], allows scaling to large datasets by trading some recall for speed and storage efficiency. Key tuning parameters include the number of clusters and the number of clusters probed at search time.

Disk-Based ANN (e.g., DiskANN). For datasets too large to fit in memory, disk-based ANN methods store most vectors on SSDs while optimizing access patterns to minimize random I/O [12]. These systems often use a small in-memory structure to guide search into disk data, achieving acceptable latency even at large scales. Tiered storage designs (RAM + SSD) are common in production vector databases.

Most ANN indexes require an offline build phase (graph construction, clustering). In practice, hybrid strategies are often used, such as combining ANN retrieval with exact re-ranking of a small candidate set.

4.3 Distributed Vector Search

When a single machine can no longer handle the dataset size or query throughput, vector search must be distributed across multiple nodes. A distributed vector database partitions data across servers and coordinates queries among them. The main aspects we will cover are sharding, replication, and load balancing.

Sharding (Partitioning). The most common approach is horizontal sharding, where the vector set is split into disjoint subsets and each shard is stored on a different node. A query is typically broadcast to all shards; each shard returns its local top-k results, which are then merged to produce the global top-k. This design scales well in capacity and throughput through parallelism, but every query incurs network fan-out. In practice, this approach is used due to its simplicity and predictability. More advanced strategies (e.g., cluster-based or semantic partitioning) attempt to route queries to only a subset of shards, but these are harder to balance and maintain, especially in RAG where any document can be relevant.

Replication. Replication is used to increase throughput and provide fault tolerance. each shard can have multiple replicas stored on different nodes, enabling continued operation if a node fails. Replication improves availability, but increases storage cost. Vector databases typically allow configuring a replication factor, with common setups using two or more replicas for high availability.

Load Balancing and Coordination. A coordinator (or routing layer) is responsible for managing queries to shards or replicas and merging results. Queries are usually sent to one

replica per shard, and the coordinator combines the partial results. Load balancing ensures even utilization of replicas and can also account for geo-distribution by routing queries to nearby nodes. Overall latency is influenced by network overhead and the slowest responding shard, meaning balancing shard sizes is important.

Distributed vector search is not always required from the start. Single-node setups can handle millions of vectors efficiently, but as datasets grow to tens or hundreds of millions, distribution becomes important. Modern vector databases abstract much of this complexity, allowing RAG systems to scale by adding nodes while maintaining low query latency.

4.4 Popular Systems

Many vector databases and libraries are commonly used to implement retrieval in RAG systems.

FAISS. FAISS is a high-performance ANN library developed by Facebook AI Research. It is a C++/Python library rather than a full database server and is often embedded directly into applications. FAISS provides efficient implementations of many ANN methods (e.g., IVF, PQ, HNSW) and supports GPU acceleration. It is widely used in research and custom production pipelines where teams want maximum control and performance. However, FAISS itself is single-node, and sharding or replication must be handled externally.

Milvus. Milvus is an open-source distributed vector database designed to manage embeddings. It provides a database-style interface. Milvus supports multiple ANN index types and can be deployed on-prem or in the cloud, often on Kubernetes. It abstracts much of the complexity of index management while still allowing flexibility.

Pinecone. Pinecone is a fully managed, cloud-native vector database offered as a service. Users interact with it through an API. It emphasizes ease of use and operational simplicity, and supports metadata filtering and large-scale deployment. The main trade-offs are reduced control over internals and costs that scale with usage.

Weaviate. Weaviate is an open-source vector database with a focus on semantic search features. It supports hybrid search (combining dense vectors with keyword-based methods like BM25), schema-aware data objects, and optional modules for automatic embedding generation. Weaviate can be self-hosted or used in a managed setup and is often chosen for its flexibility in RAG pipelines.

Other systems are also available. Elasticsearch and OpenSearch support ANN search on dense vectors, which is interesting when integrating with existing search infrastructure. Libraries such as Annoy and ScaNN offer lightweight ANN implementations, while cloud providers offer managed vector search as part of their data platforms.

In practice, the choice of system depends on scale, operational constraints, and required features. Lightweight or research setups often use FAISS directly, while production RAG systems typically rely on distributed or managed vector databases.

5 Data Engineering Challenges in RAG Systems

The challenges include how documents are preprocessed, how embeddings are managed over time, and how the system stays up to date as data change. These choices affect retrieval quality and operational cost.

5.1 Document Chunking Strategies

Chunking refers to splitting documents into smaller units suitable for retrieval. Chunking strategy strongly influences retrieval quality and index size in RAG systems.

Fixed-size vs Semantic Chunking. A basic approach is fixed-size chunking, where the text is split into blocks of a fixed length. This is easy to implement but it can break sentences or split related content. Semantic chunking instead follows document structure, such as paragraphs, sections, or question–answer pairs. These chunks are more self-contained and meaningful, improving retrieval precision, but they vary widely in size unless constraints are enforced.

Chunk Size and Overlap. Chunk size involves a trade-off between context and precision. Very large chunks dilute relevance, while very small chunks lose necessary context. In practice, chunk sizes of a few hundred tokens are common, as they fit well into LLM prompts alongside multiple retrieved passages. Overlapping chunks are often used to avoid boundary issues, improving recall, but increasing the number of chunks

Impact on Retrieval and Index Size. Chunking determines the quality of embeddings and the size of the vector index. Finer chunking produces more vectors, increasing memory usage and retrieval cost, but can improve relevance. Coarser chunking reduces index size but harm retrieval accuracy. Effective chunking means that retrieved chunks are both relevant and compact.

Content-Aware Chunking. Advanced strategies use semantic signals to define chunk boundaries, such as detecting topic shifts or using document structure. Some pipelines adapt chunking to the domain (e.g., section-based chunking for manuals or slide-based chunking for presentations). These approaches are more complex but can significantly improve retrieval quality in structured documents.

Tuning of chunk size, overlap, and segmentation strategy yields larger gains in answer quality than changes to models or retrieval algorithms.

5.2 Embedding Lifecycle Management

Embeddings are the representation used for retrieval in RAG systems, but they introduce several challenges about their lifecycle.

Embedding Model Updates. Over time, the embedding model can be upgraded or replaced with a better version. Since embeddings from different models live in different vector spaces, they cannot be mixed in the same index. As a result, changing the embedding model requires re-embedding and re-indexing the entire corpus, which is expensive at scale. Common strategies include maintaining old and new indexes in parallel during transition,

re-embedding data in phases, and versioning embeddings and indexes to allow safe rollouts and rollback.

Data Drift and Embedding Refresh. Even without model changes, embeddings become less effective as data changes and new concepts appear. Monitoring retrieval quality over time help detect when embeddings need to be refreshed or the model needs retraining.

Handling Updates and Deletions. When documents are added, modified, or removed, their embeddings must treated accordingly. Vector databases differ in how efficiently they support these operations, and updates may require index rebuilds.

Embedding Generation at Scale. Initial embedding of large corpora is done offline, while new data is embedded continuously through a streaming pipeline. This requires reliable compute resources and coordination with the vector database. Embedding generation is usually kept separate from storage to keep the system modular.

Data Quality and Preprocessing. Embedding quality depends on input text quality. Cleaning documents before embedding improves retrieval performance.

5.3 Knowledge Freshness

RAG systems have the ability to provide up-to-date information without retraining the LLM. This only holds if the underlying knowledge base is maintained.

Continuous Ingestion. In many applications, new data arrives regularly (e.g., news, documentation updates, logs). A RAG system should ingest new content quickly so it becomes searchable with minimal delay. This is handled through streaming pipelines or short micro-batches, depending on how close to real-time the application needs to be.

Deletions and Corrections. Freshness also requires removing or updating outdated or incorrect information. When documents are modified or deleted, their corresponding embeddings must be updated. In distributed systems, we have to ensure these updates propagate correctly.

Managing Staleness. Old information can accumulate if not monitored, even with regular updates. Many systems attach timestamps to documents and use ranking to prefer recent content. This reduce the chance of retrieving outdated knowledge, especially for time sensitive query.

Operational Considerations. At a scale, maintaining freshness requires pipelines, index maintenance, and monitoring data. Some systems periodically rebuild indexes to clean up fragmentation caused by updates. In practice, this is often domain-specific: some data can be updated daily, while other sources require near real-time ingestion.

A well developped RAG system treats data updates as a main concern, ensuring retrieval reflects the most current information available.

6 Query Processing, Latency, and Cost Optimization

A RAG system must balance speed, quality, and cost when queried. Even if retrieval and generation work correctly, poor query-time design can make the system slow or expensive to operate at scale.

6.1 Retrieval Latency Bottlenecks

End-to-end query latency in RAG comes from several sources.

Network Overhead. Distributed components introduce delays over the network. Co-locating services and sending minimal data help reduce latency.

Index Search Time. Vector search latency depends on the index type, parameters, and system load. ANN indexes must be tuned to balance recall and speed, and parallelism or additional hardware is often needed to handle big query rates.

Reranking and Post-processing. Optional reranking stages (e.g., cross-encoders) can significantly improve relevance but add some latency. In interactive systems, reranking is often limited to a small candidate set or omitted entirely.

LLM Generation Time. LLM inference is usually the dominant source of latency. Prompt size, output length, and model size all affect response time. Keeping prompts concise and limiting generation length are simple but effective optimizations.

6.2 Hybrid Retrieval

Hybrid retrieval combines dense and sparse search to improve recall and robustness [13]. Dense retrieval captures semantic similarity, while sparse methods like BM25 handle exact matches and rare terms.

Hybrid systems are typically implemented in one of those three ways: combining scores from dense and sparse searches, using one method to filter candidates before applying the other, or querying an index that supports both signals. This combination reduces failures cases where one method alone would miss important documents.

In RAG pipelines, hybrid retrieval improves answer reliability by making sure both semantically related passages and exact keyword matches are available to the LLM. The main trade-off are added system complexity and slightly higher query cost, which are usually acceptable for improved retrieval quality.

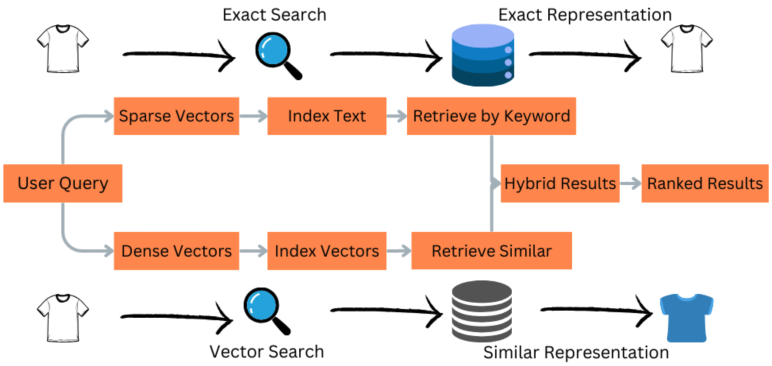

Figure 4: Example of a hybrid retrieval architecture. A user query is processed in parallel by a sparse keyword-based retrieval pipeline (top) and a dense vector-based similarity retrieval pipeline (bottom). The results from both retrievers are merged and re-ranked to produce a unified ranked result set for downstream consumption by the language model.

6.3 Cost Tradeoffs

Running a RAG involves balancing answer quality with operational cost and latency. Most costs come from LLM usage, compute resources, storage, and infrastructure choices.

LLM Token Costs. When using paid LLM APIs, cost is directly tied to input and output tokens. RAG increases input size by adding context, so limiting the size and number of retrieved chunks is critical. In practice, including only the top few most relevant passages yields savings with very small quality loss. Some systems also send simple queries to cheaper models and reserve expensive models for harder cases.

Compute Costs (CPU/GPU). RAG systems use GPUs only for generation and rely on CPUs for retrieval and indexing. Batching requests, quantization, and using smaller models can reduce cost drastically.

Storage vs Performance. Keeping all embeddings in memory is fast but costly. Disk-based indexes or compressed representations (e.g., product quantization) reduce memory cost at the expense of slightly higher latency. Storing multiple index versions also increases cost and should be done selectively.

Caching and Batching. Caching retrieval results reduce computation and batching requests improves throughput and lowers per-query cost.

Managed vs Self-Hosted Systems. Managed vector databases and LLM APIs reduce developpement efforts but introduce direct costs. Self-hosting can be cheaper at large scale but requires more operational complexity. The right choice depends on workload size and team expertise.

Overall, cost optimization in RAG is an ongoing process. By tuning context size, model choice, indexing strategy, and infrastructure scaling, engineers can reduce cost while maintaining performance and answer quality.

7 Evaluation Metrics and Benchmarking

Evaluation typically focuses on retrieval quality, end-to-end answer quality, and an understanding of what current benchmarks fail to capture.

7.1 Retrieval Metrics

Retrieval evaluation borrows standard metrics from information retrieval.

Recall@K measures whether at least one relevant document appears in the top-K retrieved results. This is the most important metric for RAG, since the LLM cannot answer correctly if the needed information is not retrieved. High recall is usually prioritized over precision.

MRR (Mean Reciprocal Rank) captures how high the first relevant document appears in the ranking. A higher MRR means that relevant information is surfaced early, which is beneficial when only a few chunks are provided to the LLM.

Precision@K measures how many of the results are relevant. While useful, it is often secondary in RAG, since the LLM can tolerate some irrelevant context as long as the correct information is present.

From a systems perspective, **latency metrics** (median, p95) and **throughput** are also important to ensure retrieval performance is consistent under load.

7.2 End-to-End RAG Evaluation

Evaluation must focus on the correctness and grounding of generated answers.

Answer correctness is measured using QA-style metrics such as : exact match or token-level F1, though free-form generation often requires some human judgment or a more flexible scoring.

Faithfulness (groundedness) evaluates whether answers are supported by the retrieved documents rather than hallucinated [14]. This can be checked through citation accuracy, human evaluation, or automated model-based checks.

Relevance and usefulness assess whether the answer actually addresses the user's question. These are often evaluated manually or with approximate text similarity metrics, though they remain difficult to measure automatically.

Freshness is a RAG-specific concern: testing whether the system can correctly answer questions about information added after the base LLM's training cutoff.

7.3 Limitations of Current Benchmarks

Most benchmarks are static and do not fully reflect real-world RAG usage [15]. They often rely on fixed texts, avoid unanswerable queries, and focus mainly on factual accuracy rather than usefulness or freshness. As a result, benchmark performance do not translate directly to production quality.

In practice, benchmarks are best used as a reference. Reliable evaluation of a RAG system usually requires custom test sets based on real user queries and domain-specific requirements, combined with ongoing monitoring after deployment.

8 Limitations and Open Challenges

Despite its strengths, RAG is not a complete solution and comes with some important limitations.

Retrieval Errors and Hallucinations. If relevant information is not retrieved, the LLM may still generate an answer based on incomplete or misleading context. Current models are not reliable when evidence is missing, which can lead to incorrect outputs.

Scalability vs Accuracy. Scaling to very large corpora requires approximate retrieval and compression, which reduce recall and introduce noise. More data also increases the chance of conflicting or outdated information being retrieved, making reasoning harder for the LLM.

Knowledge Drift and Maintenance. As facts change, old and new information can coexist in the index. Without time-awareness or pruning, systems risk citing obsolete data. Keeping knowledge clean and current remains both an engineering and research challenge.

Security and Access Control. RAG systems can expose sensible information if retrieval is not permission-aware. Preventing data leakage and handling prompt injection are important and non-trivial in large-scale deployments.

Beyond Text. Extending RAG to images, tables, audio, or video is still difficult. Multimodal indexing, retrieval, and reasoning pipelines are active research areas with no standard solution yet.

9 Future Outlook

RAG is likely to become a core part of AI infrastructure over the next decade.

RAG as Data Infrastructure. Vector search and retrieval are expected to become a main components of enterprise data systems, much like databases today. Natural language queries backed by retrieval may replace many traditional query interfaces.

Integration with Structured Knowledge. Future systems will probably combine unstructured retrieval with structured data and knowledge graphs, allowing fact lookup with semantic reasoning.

Self-Updating Systems. RAG pipelines may become autonomous, continuously ingesting new data, detecting knowledge gaps, and refreshing indexes with minimal human intervention.

Long-Term Impact. RAG has the potential to reshape enterprise search, scientific research and education by providing grounded, up-to-date answers with explicit sources.

10 Conclusion

Retrieval-Augmented Generation shifts AI question answering from being purely a modeling task to one that is just as much about data systems. Instead of relying on parametric knowledge, RAG treats knowledge as something to be stored, searched, and updated explicitly.

Throughout this review, we saw that building effective RAG systems requires strong data pipelines, scalable vector search, careful query-time optimization, and continuous evaluation. Many of the hardest challenges lie not in the LLM itself, but in retrieval quality, system maintenance, cost control, and security.

RAG offers a practical approach to improving factual grounding in large language model outputs. As vector databases, retrieval algorithms, and system architectures continue to improve, RAG is likely to become a standard design pattern for AI systems that need to reason over large, dynamic knowledge bases. The future of reliable AI is not models alone, but models tightly integrated with well-engineered data systems.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and Sebastian Riedel. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020. URL https://arxiv.org/abs/2005.11401.

[2] K2view. What is retrieval-augmented generation (rag)? https://www.k2view.com/what-is-retrieval-augmented-generation, 2024. Accessed: December 12, 2025.

[3] Pinecone. Retrieval-augmented generation (rag). https://www.pinecone.io/learn/retrieval-augmented-generation/, June 12 2025. Accessed: December 12, 2025.

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, Tianle Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023. URL https://arxiv.org/abs/2202.03629.

[5] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *EMNLP*, 2019. URL https://arxiv.org/abs/1908.10084.

[6] Microsoft Corporation. Retrieval augmented generation (rag) in azure ai search. https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview?tabs=docs, 2025. Accessed: December 12, 2025.

[7] Milvus. What is the difference between sparse and dense retrieval? https://milvus.io/ai-quick-reference/what-is-the-difference-between-sparse-and-dense-retrieval, 2025. Accessed: December 12, 2025.

[8] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is nearest neighbor meaningful? 1999. URL https://dl.acm.org/doi/10.1145/645337.650071.

[9] Jingdong Wang, Ting Zhang, and Jingkuan Song. A survey on approximate nearest neighbor search. *IEEE Transactions on Knowledge and Data Engineering*, 2021. URL https://arxiv.org/abs/2007.00819.

[10] Yu A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 2018. URL https://arxiv.org/abs/1603.09320.

[11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 2019. URL https://arxiv.org/abs/1702.08734.

[12] Suhas Jayaram Subramanya, F. Devvrit, and Harsha Vardhan Simhadri. Diskann: Fast accurate billion-point nearest neighbor search on a single node. In *NeurIPS*, 2019. URL https://arxiv.org/abs/1908.10396.

[13] Jimmy Lin, Rodrigo Nogueira, and Andrew Yates. Pretrained transformers for text ranking: Bert and beyond. *ACM SIGIR Forum*, 2021. URL https://arxiv.org/abs/2010.06467.

[14] Hannah Rashkin et al. Measuring faithfulness in retrieval-augmented generation. *arXiv*, 2023. URL https://arxiv.org/abs/2303.08896.

[15] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. *ICML*, 2020. URL https://arxiv.org/abs/2002.08909.